

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Modelagem e Simulação de FSM para Configuração de Relógio Digital na placa DE2-115

Hiago Ritter Santos

15 de junho de 2026

Sumário

1. Introdução	3
2. Descrição da implementação	4
3. Análise de código VHDL	6
3.1. Código da FSM	6
3.2. Código top timer	7
4. Análise dos testes da máquina	12
4.1. Testes sinais de hora	12
4.2. Testes sinais de minuto	12
4.3. Testes sinais de segundos	13
4.4. Conclusão dos testes	13
5. Pinagem dos componentes	14

1. Introdução

Esse relatório demonstra o processo de desenvolvimento de uma máquina de estado implementada em VHDL de um sistema de controle de timer no kit de desenvolvimento DE2-115. Nesse relatório está contido os passos que foram tomados para o desenvolvimento, testes e resultados obtidos.

2. Descrição da implementação

Para um projeto FPGA iniciamos identificando as entradas e saídas do sistema. Essa parte é necessária para identificar como o hardware vai se comportar com os sinais de controle geral do sistema, como irá se comportar em relação a entradas do usuário ou de outra máquina. Com isso em mente, identificamos que seria, necessários 4 push-buttons, identificados como KEY e vai de 0 a 3, essa parte será responsável pela interação usuário máquina para o ajuste do horário do relógio. Também foi identificado o sinal de reset que vem da KEY[0], e o clock geral do sistema, ambos esses sinais não devem ter lógicas envolvidas, devem ser sinais limpos e sem alteração, já que são essenciais para o controle do sistema. Outro sinal importante para o desenvolvimento do projeto seria a entradas das horas registradas no timer e a saída que a máquina de estado iria devolver para ela com os sinais de output. Finalmente, para criar a sensação do ajuste e estar piscando o selecionado, temos os 3 sinais para piscar o componente que está selecionado.

Após a identificação dos sinais, parti para a fase inicial do projeto, o diagrama de estados. O diagrama está disposto abaixo junto com um tabela que explica o que é cada sinal e como eles se comporta de acordo com o hardware e com a transição de cada estado. Está implementado um diagrama de Moore que cada estado tem sua saída, diferente dos diagramas Mealy que contém a saída junto com a transição de estado.

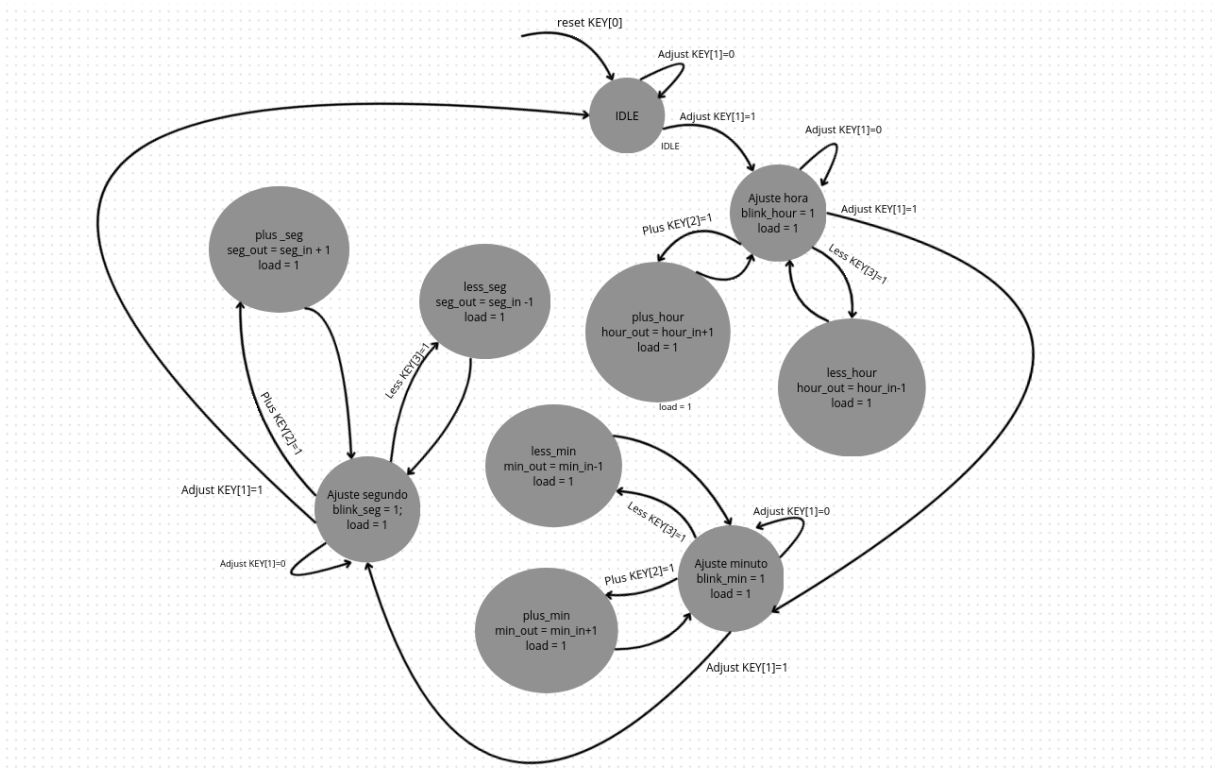


Figura 1: Diagrama de estados da máquina

- **KEY[0] (Reset):** Atua como um reset assíncrono, forçando o sistema a retornar imediatamente ao estado de operação normal (IDLE), independentemente do modo de configuração ativo.
- **KEY[1] (Modo/Navegação):** Responsável por ditar a transição sequencial entre os modos principais da máquina. É o quem faz o sistema entrar no ajuste de hora, saltar para o minuto, saltar para o segundo e, por fim, finalizar a configuração do relógio.
- **KEY[2] (Incremento):** Sinaliza a intenção de incrementar o valor do parâmetro registrado (hora, minuto ou segundo).
- **KEY[3] (Decremento):** Sinaliza a intenção de decrementar o valor do parâmetro registrado.

Estado	Descrição do Comportamento e Sinais Ativos
IDLE	Estado de espera (operação normal do relógio). Aguarda o pressionamento de KEY[1] para iniciar a o ajuste do horário.
Ajuste hora	Modo de edição da hora. Ativa o sinal <code>blink_hour = 1</code> para retorno visual na placa. Transita para estados de soma/subtração via KEY[2] e KEY[3], ou avança para os minutos via KEY[1].
plus_hour / less_hour	Estados aritméticos. Realizam a operação $hour_out = hour_in \pm 1$ e ativam o sinal <code>load = 1</code> por um ciclo de clock para carregar o novo valor no registrador da hora.
Ajuste minuto	Modo de edição dos minutos. Ativa o sinal <code>blink_min = 1</code> . Transita para os estados aritméticos de minutos ou avança para os segundos via KEY[1].
plus_min / less_min	Estados aritméticos. Realizam a operação $min_out = min_in \pm 1$ e ativam o sinal <code>load = 1</code> para atualizar o registrador dos minutos.
Ajuste segundo	Modo de edição dos segundos. Ativa o sinal <code>blink_seg = 1</code> . Permite incremento/decremento e, ao pressionar KEY[1], encerra a configuração retornando ao IDLE.
plus_seg / less_seg	Estados aritméticos. Realizam a operação $seg_out = seg_in \pm 1$ e ativam o sinal <code>load = 1</code> para atualizar o registrador dos segundos.

Tabela 1: Descrição funcional dos estados da FSM.

3. Análise de código VHDL

3.1. Código da FSM

A entidade `fsm_relogio` define a interface da máquina de estados, mapeando os botões de controle (KEY) aos sinais internos de navegação e operação.

VHDL

```
1
2  entity fsm_relogio is
3      port (
4          clk, reset : in std_logic;
5          adjust      : in std_logic; -- KEY[1]
6          plus, less  : in std_logic; -- KEY[2], KEY[3]
7
8          -- Entradas de dados do timer
9          hour_in     : in std_logic_vector(4 downto 0);
10         min_in      : in std_logic_vector(5 downto 0);
11         seg_in       : in std_logic_vector(5 downto 0);
12
13         -- Saídas de controle e valores ajustados
14         blink_h, blink_m, blink_s : out std_logic;
15         load              : out std_logic;
16         hour_out          : out std_logic_vector(4 downto 0);
17         min_out           : out std_logic_vector(5 downto 0);
18         seg_out           : out std_logic_vector(5 downto 0)
19     );
20 end fsm_relogio;
21
```

Listagem 1: Definição da entidade da FSM de controle do relógio.

A entidade expõe os sinais `hour_in`, `min_in` e `seg_in` para que a máquina monitore o estado atual do timer, e os sinais `out` para que o novo valor configurado seja carregado no registrador principal quando o sinal `load` for ativado.

Após a declaração da entidade, implementamos a arquitetura com dois `Process`, um para `reset` e segurança caso a máquina de estado não saiba para qual estado ir, vai para o estado `IDLE` padrão e segue o funcionamento a partir daquele estado. E também com o `Process` com os estado implementados, abaixo só terá um exemplo para incremento e decremento da hora.

```

1
2  case state is
3      when IDLE => if adjust = '1' then next_state <= AJUSTE_HORA; end if;
4
5      when AJUSTE_HORA =>
6          blink_h <= '1';
7          load <= '1';
8          if plus = '1' then next_state <= PLUS_HOUR;
9          elsif less = '1' then next_state <= LESS_HOUR;
10         elsif adjust = '1' then next_state <= AJUSTE_MINUTO;
11         end if;
12
13         when PLUS_HOUR =>
14             blink_h <= '1';
15             load <= '1';
16             next_state <= AJUSTE_HORA;
17             -- limite: Se for 23, vai para 0
18             if unsigned(hour_in) = 23 then
19                 hour_out <= std_logic_vector(to_unsigned(0, 5));
20             else
21                 hour_out <= std_logic_vector(unsigned(hour_in) + 1);
22             end if;
23
24         when LESS_HOUR =>
25             blink_h <= '1';
26             load <= '1';
27             next_state <= AJUSTE_HORA;
28             -- limite: Se for 0, vai para 23
29             if unsigned(hour_in) = 0 then
30                 hour_out <= std_logic_vector(to_unsigned(23, 5));
31             else
32                 hour_out <= std_logic_vector(unsigned(hour_in) - 1);
33             end if;
34
35         -- continua também para os próximos estados
36         -- deve-se utilizar sempre as limitações dos horários
37

```

3.2. Código top timer

Esse código implementa o contador geral do sistema e instância todos os componentes necessários para o funcionamento do projeto. Inicialmente, precisamos declarar dentro da arquitetura do timer todos os componentes que serão utilizados, bem como os sinais de comunicação entre esses componentes. O professor já disponibilizou para o projeto um código faz o sincronismo entre os push buttons e a placa para poder incrementar e decrementar de forma adequada devido ao clock da placa passar por várias vezes em um tempo muito curto.

```
1
2  -- declaracao da fsm_relogio como componente
3  component fsm_relogio is
4      port (
5          clk, reset, adjust, plus, less : in std_logic;
6          hour_in                        : in std_logic_vector(4 downto 0);
7          min_in, seg_in                : in std_logic_vector(5 downto 0);
8          blink_h, blink_m, blink_s    : out std_logic;
9          load                          : out std_logic;
10         hour_out                      : out std_logic_vector(4 downto 0);
11         min_out, seg_out              : out std_logic_vector(5 downto 0)
12     );
13 end component;
14
15 -- declarando o sincronizador de botões como componente
16 component sync_keys is
17     port(
18         clk, reset : in std_logic;
19         keys_i     : in std_logic_vector(2 downto 0);
20         keys_o     : out std_logic_vector(2 downto 0)
21     );
22 end component;
23
```

Listagem 3: Definição da entidade da FSM de controle do relógio.

Após instanciar os componentes necessários, também é necessário implementar os sinais utilizados para conectar os componetes e saber o que cada um irá fazer no sistema


```

1
2   -- declarações de sinais internos
3   signal reset : std_logic;
4   -- sinais dos Botoes
5   signal raw_keys : std_logic_vector(2 downto 0); -- leitura dos botoes
6   signal syncro_keys : std_logic_vector(2 downto 0); -- leitura dos
   botoes sincronizados
7   signal adjust_sig : std_logic; -- sinal de ajuste para prox estado
8   signal plus_sig : std_logic; -- incrementa
9   signal less_sig : std_logic; -- decrementa
10  -- horario atual que vai vir de timer
11  signal hour_current : std_logic_vector(4 downto 0);
12  signal min_current  : std_logic_vector(5 downto 0);
13  signal sec_current  : std_logic_vector(5 downto 0);
14  -- sinais de ajuste (saida da FSM -> entrada do timer)
15  signal fsm_hour_out : std_logic_vector(4 downto 0);
16  signal fsm_min_out  : std_logic_vector(5 downto 0);
17  signal fsm_sec_out  : std_logic_vector(5 downto 0);
18  signal fsm_load     : std_logic;
19  -- sinais de blink
20  signal blink_h, blink_m, blink_s : std_logic;
21  signal fsm_any_blink : std_logic;
22  signal blink_oscillator : std_logic;
23  -- bcd para o display
24  signal hourT, hourU : std_logic_vector(3 downto 0);
25  signal minT, minU   : std_logic_vector(3 downto 0);
26  signal secT, secU   : std_logic_vector(3 downto 0);
27
28  signal hourT_blink, hourU_blink : std_logic_vector(3 downto 0);
29  signal minT_blink, minU_blink : std_logic_vector(3 downto 0);
30  signal secT_blink, secU_blink : std_logic_vector(3 downto 0);
31

```

Listagem 4: Definição da entidade da FSM de controle do relógio.

Após finalizar as instanciações e declarações de sinais entre os componentes, é necessário fazer as conexões do circuito via Port Map, nesse estágio que vamos o conectar todos os componentes e finalizar a conexão.

Abaixo está um exemplo de como fazemos a conexão entre os sinais declarados e os componentes. Ou seja, os sinais de saída de um entram no sinais de entrada de outro e assim sucessivamente até estar tudo conectado de maneira adequada.

VHDL

```
1
2  -- instanciando componente da fsm_relogio
3  inst_fsm: fsm_relogio
4  port map(
5      clk      => CLOCK_50,
6      reset    => reset,
7      adjust   => adjust_sig,
8      plus     => plus_sig,
9      less     => less_sig,
10
11     -- fsm lê a hora atual do timer
12     hour_in  => hour_current,
13     min_in   => min_current,
14     seg_in   => sec_current,
15
16     -- fsm controla os displays e o carregamento do Timer
17     blink_h  => blink_h,
18     blink_m  => blink_m,
19     blink_s  => blink_s,
20     load     => fsm_load,
21
22     -- fsm devolve a hora adjusted de volta pro timer
23     hour_out => fsm_hour_out,
24     min_out  => fsm_min_out,
25     seg_out  => fsm_sec_out
26
```

Listagem 5: Definição da entidade da FSM de controle do relógio.

É necessário gerenciar como será o piscar dos componentes selecionados, por isso utilizamos uma porta OR que faz o controle, se teve algum sinal para piscar (blink) vindo da FSM, vamos habilitar o sinal global e fazer o tratamento individual nos componentes responsáveis do blink de cada um.

VHDL

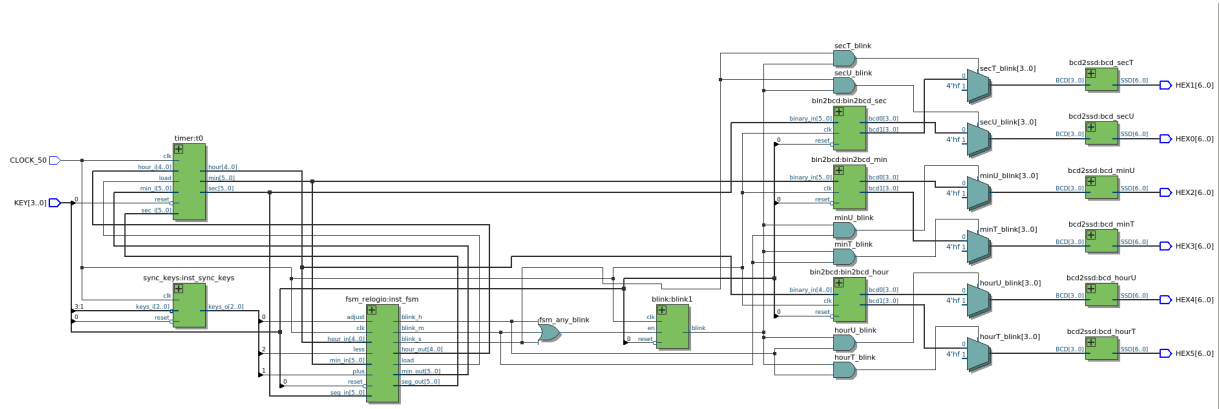
```

1
2  -- pisca se um dos três estiver mandando piscar
3  fsm_any_blink <= blink_h or blink_m or blink_s;
4
5  blink1: blink
6  port map(
7    clk    => CLOCK_50,
8    reset  => reset,
9    en     => fsm_any_blink,
10   blink  => blink_oscillator
11 );
12  secU_blink <= (others=>'1') when (blink_oscillator='1' and blink_s='1')
   else secU;
13  secT_blink <= (others=>'1') when (blink_oscillator='1' and blink_s='1')
   else secT;
14  minU_blink <= (others=>'1') when (blink_oscillator='1' and blink_m='1')
   else minU;
15  minT_blink <= (others=>'1') when (blink_oscillator='1' and blink_m='1')
   else minT;
16  hourU_blink <= (others=>'1') when (blink_oscillator='1' and
   blink_h='1') else hourU;
17  hourT_blink <= (others=>'1') when (blink_oscillator='1' and
   blink_h='1') else hourT;
18

```

Listagem 6: Definição da entidade da FSM de controle do relógio.

Após toda configuração necessária feita conforme demonstrado acima, vamos ter um circuito totalmente integrado entre si e com cada componente devidamente conectado, portanto assim, a parte de desenvolvimento do código e da lógica dos estados está finalizada. Agora vamos ter um circuito completo, abaixo tem uma representação RTL do circuito gerado.

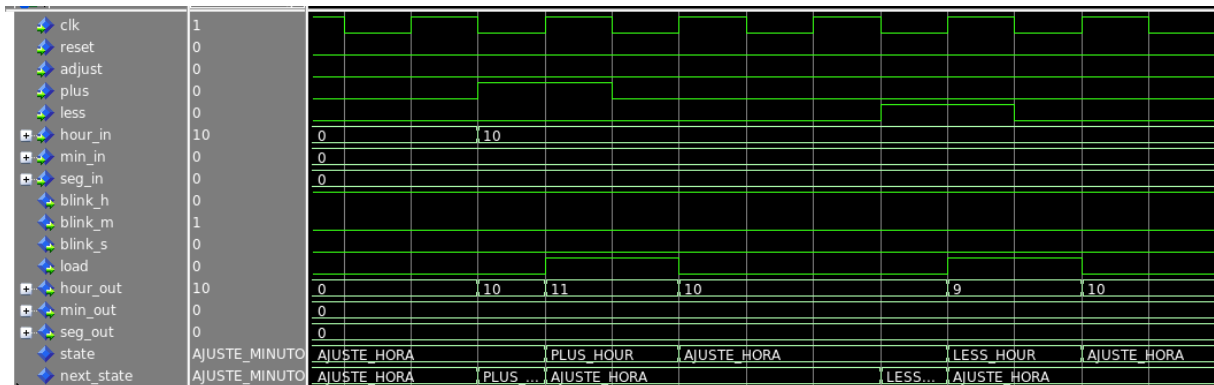


4. Análise dos testes da máquina

Para testar o funcionamento da máquina desenvolvida, utilizamos o ModelSim para conseguir executar um script com todos os testes necessários antes de embarcar o código na placa.

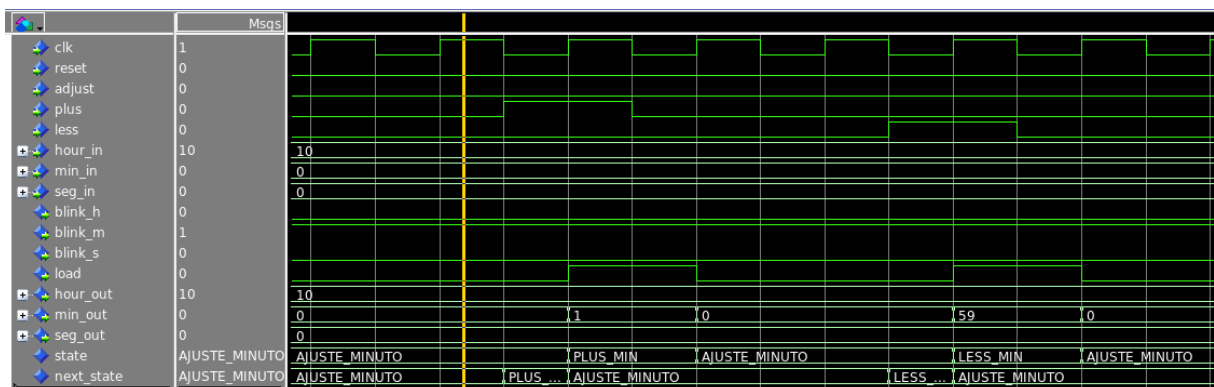
Foi testado as situações de extremos da contagem de hora, minutos, segundo, testamos a transição entre estados e o funcionamento do sinal de piscar para cada estado, se estaria de acordo com o necessário para o circuito.

4.1. Testes sinais de hora



Nesse teste forçamos o valor 10 para hora e observa-se que a máquina está passando do estado ajuste hora e indo para o estado de adição logo na borda de subida do clock. No sinal de retorno que é hour_out também já aparece o retorno correto que é 11 para a entrada 10. Um pouco mais a frente também se observa o valor 10 forçado na entrada, a transição para o estado de decremento de hora e o valor 9 aparecendo no sinal de retorno de output da máquina, demonstrando que essa parte da máquina de estado funciona de maneira adequada. Esse resultado foi obtido depois de realizar o mesmo teste e identificar que nas bordas, valor 24 e valor 0 de entrada estava retornando um valor fora do intervalo correto para as horas, após a correção no código VHDL a máquina começou a apresentar comportamento correto.

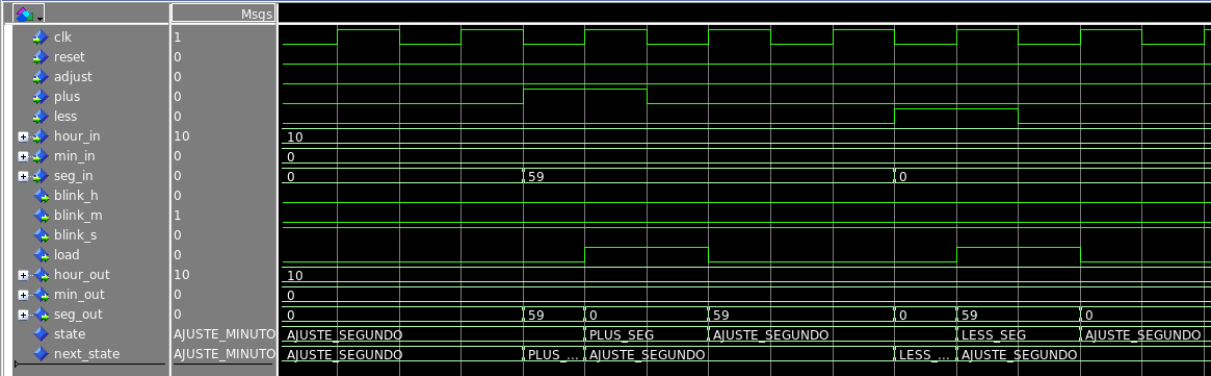
4.2. Testes sinais de minuto



Nesse teste foram testados as bordas que poderiam dar problema como no teste das horas, o valor forçado para a entrada dos estados é o 0 e podemos observar o comportamento ao incrementar e decrementar o valor. No primeiro teste o valor sai de 0 e vai para 1 no estado de incremento, logo após, aparece o mesmo valor sendo decrementado e retornando o valor 59, correto esperado para o comportamento da máquina. Resultado obtido também após testes de

extremos retonarem resultados incorretos ao incrementar e decrementar valores das bordas do intervalo.

4.3. Testes sinais de segundos



Segue com a mesma lógica dos testes anteriores, forçamos o 59 para o incremento e verificamos que está de fato retornando o valor 0 esperado para a contagem do relógio. Bem como para a entrada 0 no estado de decremento retonar o 59, indicando que os intervalos estão sendo respeitados.

4.4. Conclusão dos testes

Como a máquina de estado em si apenas trata as entradas e saídas do que está registrado no timer, a máquina precisa ser capaz de ler e retornar os valores corretos para o componente do timer, o qual estáorquestrando a contagem do tempo e também tem um sinal de load conectado na máquina de estado para interromper a contagem assim que estiver nos estados de ajuste. Ou seja, com os testes acima, já conseguimos observar que a máquina tem capacidade de retornar valores corretos para o sistema, tem a capacidade de verificar sinais externos e retornar valores de acordo com as entradas dadas.

5. Pinagem dos componentes

Utilizamos o kit de desenvolvimento DE2-115, da família cyclone IV, device EP4CE115F29C7. Para poder mostrar os números na placa, foi necessário designar os pinos de entrada e saída para a placa conseguir mostrar o que queríamos, portanto, para as entradas, as Keys utilizadas no código, utilizamos os seguintes pinos da placa

Signal Name	FPGA Pin No.	Description
KEY[0]	PIN_M23	Push-button[0]
KEY[1]	PIN_M21	Push-button[1]
KEY[2]	PIN_N21	Push-button[2]
KEY[3]	PIN_R24	Push-button[3]

A tabela acima mostra os pinos dos botões, os outros pinos já estavam disponíveis no .QAR disponibilizado para iniciar o projeto. Os demais pinos podem ser consultados na documentação disponibilizada na wiki da engenharia do campus São José.¹

¹Veja a lista completa na Wiki do IFSC SJ (Pinagem do kit DE2-115).